

PRÁCTICA_6: MÓDULOS

Módulos

1. Crea un módulo en el que definirás dos funciones, una para validar usuarios y otra para validar contraseñas:

- El nombre de usuario debe contener un mínimo de 6 caracteres y un máximo de 12.
- El nombre de usuario debe ser alfanumérico.
- Nombre de usuario con menos de 6 caracteres, retorna el mensaje "El nombre de usuario debe contener al menos 6 caracteres".
- Nombre de usuario con más de 12 caracteres, retorna el mensaje "El nombre de usuario no puede contener más de 12 caracteres".
- Nombre de usuario con caracteres distintos a los alfanuméricos, retorna el mensaje "El nombre de usuario puede contener solo letras y números".
- Nombre de usuario válido, retorna True.
- La contraseña debe contener un mínimo de 8 caracteres.
- Una contraseña debe contener letras minúsculas, mayúsculas, números y al menos 1 carácter no alfanumérico.
- La contraseña no puede contener espacios en blanco.
- Contraseña válida, retorna True.
- Contraseña no válida, retorna el mensaje "La contraseña elegida no es segura".

Programa que solicita usuarios y sus contraseñas y los valida usando el módulo anterior.:

- a. Pedimos usuario y lo validamos, si no es correcto lo pedimos de nuevo
- b. Pedimos contraseña, si no es correcta la pedimos de nuevo

- c. Cuando ambos datos son correctos los almacenamos en un diccionario. No se pueden repetir ni usuarios ni contraseñas.

Para finalizar pulsaremos "*" al introducir usuario

2. Crea un paquete con el módulo anterior y añade otro módulo con funciones que validan los datos personales tales como dni y edad (no inferior a 18)
3. Varia el programa anterior para almacenar usuario, contraseña, dni y edad. Los datos los almacenarás en un diccionario siendo la clave el usuario y el valor una lista con el resto de los datos
4. Distribuye e instala en tu máquina el paquete, comprueba que el script del apartado anterior lo puedes ejecutar desde cualquier ruta. Puedes intercambiar el paquete con algún compañero e instalarla cada uno el del otro.
5. Una forma de representar fechas es utilizando números enteros de la siguiente forma: los primeros cuatro dígitos corresponden al año, los siguientes dos dígitos corresponden al mes, y los últimos dos dígitos corresponden al día. Por ejemplo, el número 20200901 representa la fecha 01/09/2020.

Para esta tarea, supondremos que los números que representan fechas siempre tienen 8 dígitos, y que el primer dígito siempre es distinto a 0 (por lo que sólo sirve para representar fechas entre el año 1000 y el año 9999).

El objetivo de esta tarea es programar un módulo con funciones que permitan trabajar con fechas en este formato. Las funciones que debes implementar son las siguientes:

- a) Función year: recibe una fecha representada como entero y devuelve un texto con el año de la fecha.
- b) Función month: recibe una fecha representada como entero y devuelve un texto con el mes de la fecha. Si el número del mes no es válido, debe devolver el texto 'FECHA INCORRECTA'.
- c) Función monthName: recibe una fecha representada como entero y devuelve un texto con el nombre del mes, en español, de la fecha ('enero', 'febrero', . . . , 'diciembre'). Si el número del mes no es válido, debe devolver el texto 'FECHA INCORRECTA'.
- d) Función day: recibe una fecha representada como entero y devuelve un texto con el día de la fecha. Si el número del día no es válido (esto depende

del mes, considere si el año es bisiesto para el caso de febrero) o el mes es inválido (por lo que no se puede verificar el día), debe devolver el texto 'FECHA INCORRECTA'.

- e) Función `date2Text`: recibe una fecha representada como entero y devuelve un texto con la fecha completa en el formato 'DD/MM/AAAA'. Por ejemplo, `date2Text(20200903)` devuelve el texto '03/09/2020'. Si el mes o el día no son válidos, debe devolver el texto 'FECHA INCORRECTA'.
- f) Función `date2TextName`: recibe una fecha representada como entero y devuelve un texto con la fecha completa en el formato 'X de Y de Z', usando el nombre del mes. Por ejemplo, `date2Text(20200903)` devuelve el texto '3 de septiembre de 2020'. Si el mes o el día no son válidos, debe devolver el texto 'FECHA INCORRECTA'.
- g) Función `manyDates`: recibe un número con varias fechas representadas como entero (cantidad no definida a priori), y devuelve un texto con las fechas en el formato 'DD/MM/AAAA', con cada fecha separada por un salto de línea (carácter '\n') y en el mismo orden que aparecen en el número. Por ejemplo, `manyDates(2020090320200904)` devuelve el texto '\n03/09/2020\n04/09/2020'. Note que después de la última fecha no hay un carácter de salto de línea, pero sí en el primer carácter del texto. Si algunas de las fechas no son válidas, solo sus líneas correspondientes deben decir 'FECHA INCORRECTA'.
- h) Función `sameDates`: recibe dos parámetros, un número con varias fechas representadas como entero (cantidad no definida a priori) y una fecha representada como entero, y devuelve la cantidad de fechas iguales en el entero con varias fechas a la fecha correspondiente al segundo parámetro, pero sólo considerando el día y mes (el año puede ser distinto). Por ejemplo, `sameDates(202009032021090420000903, 21000903)` devuelve 2.

Para resolver este ejercicio usarás el módulo `FuncionesAuxiliares.py` que se te proporciona.